Digital Image Processing

Assignment 2 – Solutions

Chapter 3: Intensity Transformations & Spatial Filtering

Gonzalez & Woods, 3rd Edition

Contents

Chapter 3 at a Glance 2

1 Problem 3.4 – Bit-Plane Slicing Transformations 3

2 Problem 3.5 – Effect of Zeroing Bit Planes on the Histogram 5

3 Problem 3.6 – Why Discrete HE Does Not Yield a Flat Histogram 7

4 Problem 3.7 – Second-Pass HE = First-Pass HE 9

5 Problem 3.13 – Histograms of Arithmetic Image Combinations 11

6 Problem 3.15 – Box-Filter Moving-Average Algorithm 13

7 Problem 3.19 – Median Filter: Computation and Update 15

8 Problem 3.24 – Isotropy (Rotation Invariance) of the Laplacian 17

9 Problem 3.28 – Laplacian Subtraction is Proportional to Unsharp Masking 19

Summary of All Solutions 21

Chapter 3 at a Glance

Chapter 3 covers two broad categories of **spatial domain** image processing:

1. **Point (intensity) operations**: output at (x, y) depends only on $f(x, y)$.
   - Intensity transformations, bit-plane operations, histogram processing.
2. **Neighbourhood (spatial filter) operations**: output at (x, y) depends on a region around (x, y).
   - Smoothing (lowpass), sharpening (highpass), median filtering.

Notation used throughout: $L = 256$ (8-bit image, levels $0 \ldots 255$); $p_r(r_k) = n_k/n =$ probability of intensity level r_k; $\nabla^2 =$ Laplacian operator.

1 Problem 3.4 – Bit-Plane Slicing Transformations

Problem Statement

Propose a set of intensity-slicing transformations capable of producing all the individual bit planes of an 8-bit monochrome image. (For example, a transformation with $T(r) = 0$ for $r \in [0, 127]$ and $T(r) = 255$ for $r \in [128, 255]$ produces an image of the 8th bit plane in an 8-bit image.)

Every pixel value $r \in [0, 255]$ in an 8-bit image has a unique binary representation:

$$r = \sum_{k=0}^{7} b_k(r) \cdot 2^k, \qquad b_k(r) \in \{0, 1\}$$

where b_0 is the **LSB** (least significant) and b_7 the **MSB** (most significant). Separating the image into its 8 bit planes is useful for:

- **Image compression**: the MSBs carry most of the visual information; LSBs contribute fine detail.
- **Watermarking**: embed hidden data in LSBs without visibly altering the image.
- **Analysis**: understanding what part of each bit plane contributes to the image.

Key observation. The kth bit of r cycles with period 2^{k+1}: it is *0* for the first 2^k values of each period, then *1* for the next 2^k values.

Solution

General formula for the kth bit-plane transformation ($k = 0, 1, \ldots, 7$):

$$\boxed{T_k(r) = 255 \cdot \left(\lfloor r/2^k \rfloor \bmod 2\right)} \tag{1}$$

- $\lfloor r/2^k \rfloor$ right-shifts r by k bits.
- Taking mod 2 isolates the kth bit (0 or 1).
- Multiplying by 255 maps to the full display range $\{0, 255\}$.

Explicit piecewise-constant form: The period of T_k is $P_k = 2^{k+1}$. Within each period starting at $j \cdot 2^{k+1}$ ($j = 0, 1, \ldots$):

$$T_k(r) = \begin{cases} 0 & r \in \left[j \cdot 2^{k+1},\ j \cdot 2^{k+1} + 2^k - 1\right] \\ 255 & r \in \left[j \cdot 2^{k+1} + 2^k,\ (j+1) \cdot 2^{k+1} - 1\right] \end{cases}$$

The 8 specific transformations:

Plane k	Period	Low range (output 0)	High range (output 255)
7 (MSB)	256	[0, 127]	[128, 255]
6	128	$[0, 63] \cup [128, 191]$	$[64, 127] \cup [192, 255]$
5	64	... alternates every 32	...
4	32	... alternates every 16	...
3	16	... alternates every 8	...
2	8	... alternates every 4	...
1	4	$\{0, 1\} \cup \{4, 5\} \cup \ldots$	$\{2, 3\} \cup \{6, 7\} \cup \ldots$
0 (LSB)	2	even values	odd values

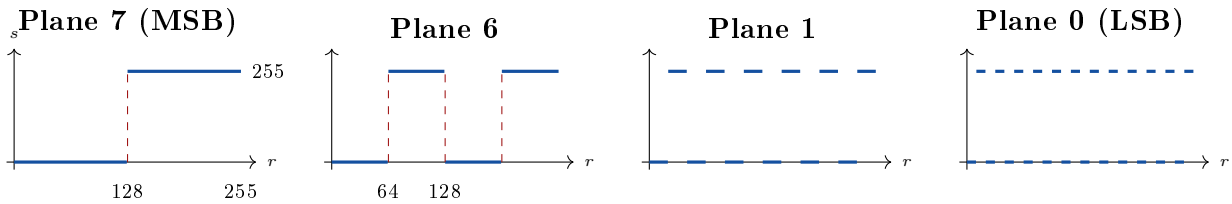


Figure 1: Bit-plane slicing transformations for planes 7, 6, 1, and 0. Each function has period 2^{k+1} and alternates between 0 and 255 every 2^k gray levels. The MSB changes only once; the LSB changes with every pixel.

2 Problem 3.5 – Effect of Zeroing Bit Planes on the Histogram

Problem Statement

(a) What effect would setting to zero the lower-order bit planes have on the histogram of an image in general?
(b) What would be the effect on the histogram if we set to zero the higher-order bit planes instead?

The histogram $h(r)$ counts how many pixels have intensity r. When we *zero bit plane* k, every pixel whose k^{th} bit is 1 has that bit cleared:

$$r' = r - b_k(r) \cdot 2^k$$

This is equivalent to applying the transformation $T_k^{\text{zero}}(r)$ that zeros the kth bit while preserving all others.

Solution

(a) Setting *lower-order* bit planes to zero.
Suppose we zero the lowest p bit planes (planes $0, 1, \ldots, p-1$). The new pixel value of r is:

$$r' = 2^p \left\lfloor \frac{r}{2^p} \right\rfloor$$

That is, each pixel is *rounded down to the nearest multiple of* 2^p.
Effect on the histogram:

1. All values $r \in [j \cdot 2^p,\ (j+1) \cdot 2^p - 1]$ for each block j are merged into a single value $j \cdot 2^p$.
2. The histogram becomes **non-zero only at multiples of** 2^p (levels $0, 2^p, 2 \cdot 2^p, \ldots$).
3. Each surviving level accumulates the counts of 2^p original levels, so typical histogram heights roughly *increase by a factor of* 2^p.
4. The number of distinct levels drops from $L = 256$ to $256/2^p$.

Example – zeroing the LSB ($p = 1$): Even-valued pixel $r = 2k$ and odd-valued $r = 2k + 1$ both map to $r' = 2k$. The histogram gets non-zero values only at even levels, each bar's height $\approx h(r) + h(r+1)$.

(b) Setting *higher-order* bit planes to zero.
Zeroing the top p bit planes (planes $7, 6, \ldots, 8 - p$) removes the most significant contribution to the pixel value.
Effect on the histogram:

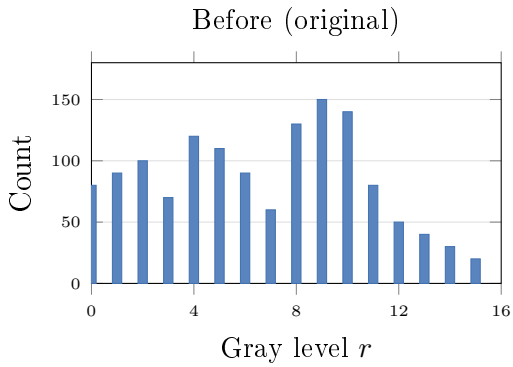
1. All pixels with those high bits set are shifted *downward* in intensity. For example, zeroing bit 7 maps $r \in [128, 255]$ to $r' = r - 128 \in [0, 127]$.

2. The histogram **compresses into the lower intensity range** $[0, 2^{8-p} - 1]$.

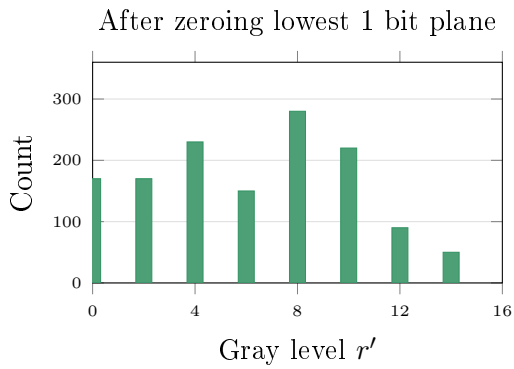
3. Levels in the upper range of the original histogram are *folded back* on top of the lower range, causing the lower portion of the histogram to increase in height.

4. The resulting image appears **much darker** on average and exhibits a *periodic repetition* of gray-level structure because the remaining bits still encode the same relative patterns.

Example – zeroing the MSB (bit 7, $p = 1$): Pixels in $[128, 255]$ all shift to $[0, 127]$. The histogram in $[0, 127]$ becomes the sum of the original histograms of $[0, 127]$ and $[128, 255]$. The upper half of the histogram ($[128, 255]$) becomes empty.



(a) Original 4-bit histogram (all levels present)



(b) Even levels only; heights $\approx$ double; gaps at odd levels

Figure 2: Effect of zeroing the LSB. Consecutive pairs $(r, r{+}1)$ merge into a single bar at even level r, and odd levels disappear.

3 Problem 3.6 – Why Discrete HE Does Not Yield a Flat Histogram

Problem Statement

Explain why the discrete histogram equalization technique does not, in general, yield a flat histogram.

Continuous case. For a continuous intensity variable r, applying the transformation $s = T(r) = (L-1)\int_0^r p_r(w)\,dw$ (the CDF) produces an output with a *perfectly uniform* PDF. This is a standard result from probability theory (the probability-integral transform).

Discrete case. For a discrete image with L levels and total n pixels, the equalization formula is:

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j) = \frac{L-1}{n}\sum_{j=0}^{k} n_j \tag{2}$$

The result s_k is a *real number* that must be **rounded** to the nearest integer to obtain a valid pixel value.

Solution

There are **two fundamental reasons** a perfectly flat histogram is impossible in the discrete case:

Reason 1 – Integer Quantisation of the CDF.
Eq. (2) gives a continuous value that must be rounded to an integer level. After rounding:

- Several consecutive input levels r_j may round to the *same* output level s_k. Their pixel counts are **merged**, creating a large spike in the output histogram.
- Conversely, some integer output levels may not correspond to any rounded s_k, leaving **empty bins** (gaps).

These merges and gaps make exact uniformity impossible.

Reason 2 – Conservation of Distinct Levels.
The transformation T cannot *create* new distinct gray levels – it can only rearrange or merge existing ones. If the input uses $M < L$ distinct levels, the output can have at most M distinct levels; the remaining $(L-M)$ output levels will be empty no matter what.

Mathematical illustration:
If an input level r_k has probability $p_r(r_k) > \frac{1}{L-1}$, then:

$$s_k - s_{k-1} = (L-1)\,p_r(r_k) > 1$$

meaning **at least two consecutive output levels are skipped** when jumping from s_{k-1} to s_k, guaranteeing empty bins. Simultaneously, if $p_r(r_k) < \frac{1}{L-1}$, the CDF

increment is < 1 and multiple input levels round to the same output level, merging their bins.

What HE does guarantee: In spite of these limitations, HE gives the *closest possible approximation* to a uniform histogram under integer constraints, and it always spreads the histogram to make better use of the available dynamic range.

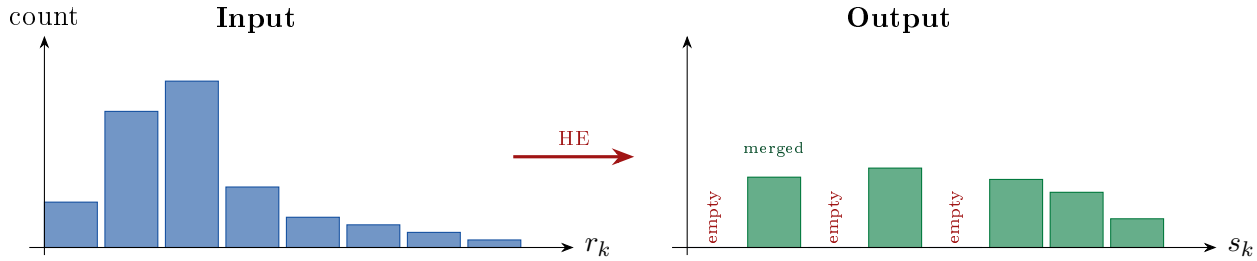


Figure 3: Discrete HE cannot eliminate the empty bins (shown in red) because several input levels merge into a single output level while other output levels receive no pixels at all.

4 Problem 3.7 – Second-Pass HE = First-Pass HE

Problem Statement

Suppose that a digital image is subjected to histogram equalization. Show that a second pass of histogram equalization (on the histogram-equalized image) will produce exactly the same result as the first pass.

An operation is called **idempotent** if applying it twice gives the same result as applying it once: $T(T(x)) = T(x)$.
Histogram equalization can be thought of as: "transform the image so that its CDF becomes linear (uniform histogram)." If the image already has a uniform histogram, the equalization transformation is the identity. After one pass of HE, the image has an approximately uniform histogram, so a second pass should leave it unchanged.

Let the **first-pass** equalization transform the input levels r_k to output levels:

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j)$$

After the first pass, the equalized image has histogram:

$$p_s(s_m) = \sum_{\{k\,:\,T(r_k)=s_m\}} p_r(r_k)$$

(All input levels that mapped to s_m have their probabilities pooled.)
Key step – CDF of the equalized image.
For any output level s_m, the CDF of the equalized image is:

$$F_s(s_m) = \sum_{l\le m} p_s(s_l) = \sum_{\{k\,:\,T(r_k)\le s_m\}} p_r(r_k) = \sum_{k=0}^{k_{\max}(m)} p_r(r_k) = F_r(r_{k_{\max}(m)}) \tag{3}$$

where $k_{\max}(m)$ is the largest input index satisfying $T(r_k) \le s_m$.
Second-pass transformation.
Applying HE to the equalized image gives:

$$s'_m = (L-1)\cdot F_s(s_m) = (L-1)\cdot F_r(r_{k_{\max}(m)}) = T(r_{k_{\max}(m)})$$

Now, by definition of $k_{\max}(m)$, the largest r_k satisfying $T(r_k) \le s_m$ is the one that *actually maps to* s_m (since T is non-decreasing). Therefore $T(r_{k_{\max}(m)}) = s_m$.
Substituting:

$$\boxed{s'_m = s_m}$$

The second-pass transformation is the **identity** – every equalized level maps to itself. Hence the second pass produces *exactly the same image* as the first pass. □

Intuition. After HE, the CDF of the equalized image is already as uniform (flat) as possible given the discrete constraints. Applying HE again attempts to "re-equalise an already-equalised histogram", which results in the identity transformation.

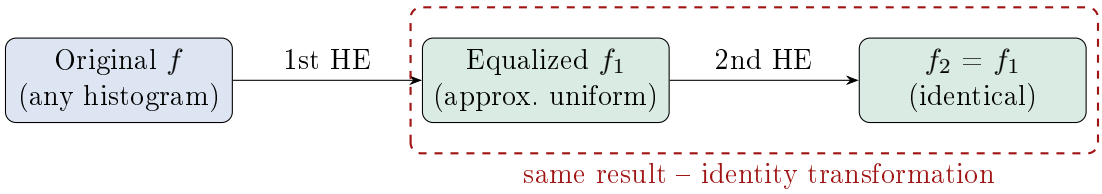


Figure 4: Histogram equalization is idempotent: the second pass produces the same result as the first pass.

5 Problem 3.13 – Histograms of Arithmetic Image Combinations

Problem Statement

Two images $f(x,y)$ and $g(x,y)$ have histograms h_f and h_g. Give the conditions under which you can determine the histograms of
(a) $f(x,y)+g(x,y)$ (b) $f(x,y)-g(x,y)$ (c) $f(x,y)\cdot g(x,y)$ (d) $f(x,y)\div g(x,y)$
in terms of h_f and h_g. Explain how to obtain the histogram in each case.

The normalised histogram $p_f(r) = h_f(r)/n$ is a **probability mass function (PMF)** for the pixel intensities. Given two images f and g, the PMF of any pixel-wise combination $c(x,y) = F(f(x,y), g(x,y))$ can be derived from the **joint PMF** $p_{f,g}(r,s)$.
Fundamental condition: the joint PMF equals the product of the marginals if and only if f and g are **statistically independent pixel-by-pixel**:

$$p_{f,g}(r,s) = p_f(r) \cdot p_g(s) \quad \Leftrightarrow \quad f \text{ and } g \text{ are pixel-wise independent}$$

Note: the histograms h_f and h_g provide only the *marginal* distributions; they do not reveal the *joint* distribution. The joint distribution is needed to compute the histogram of any combination.

Solution

Necessary condition for all four cases:
The histograms of h_f and h_g alone are sufficient to determine the combined histograms *if and only if the two images are pixel-wise statistically independent.*

(a) Sum: $h = f + g$.
Using the *discrete convolution* of the PMFs:

$$\boxed{p_{f+g}(z) = \sum_{r=0}^{z} p_f(r) \cdot p_g(z-r)}$$

This is the **discrete convolution** $p_f * p_g$. The range of values spans $[0_f + 0_g,\ (L-1)_f + (L-1)_g]$.
How to obtain: form the convolution of the two normalised histograms (e.g. using polynomial multiplication or FFT convolution).

(b) Difference: $h = f - g$.

$$\boxed{p_{f-g}(z) = \sum_{r} p_f(r) \cdot p_g(r-z)}$$

This is the **cross-correlation** of p_f and p_g, equivalently the convolution of p_f with the *reversed* $\tilde{p}_g$, where $\tilde{p}_g(s) = p_g(-s)$: i.e. $p_{f-g}(z) = p_f * \tilde{p}_g$.
The range spans $[-(L-1),\ +(L-1)]$.

(c) Product: $h = f \cdot g$.

In the continuous case with independent f, g:

$$\boxed{p_{f\cdot g}(z) = \int \frac{1}{|r|}\, p_f\!\left(\frac{z}{r}\right) p_g(r)\, dr}$$

Even with independence the calculation is more complex because it involves scaling and the Jacobian $1/|r|$. For the discrete case, summing over all pairs (r, s) with $r \cdot s = z$:

$$p_{f\cdot g}(z) = \sum_{\substack{r,s\geq 0\\ r\cdot s=z}} p_f(r)\cdot p_g(s)$$

(d) Quotient: $h = f/g$ **(assuming** $g \neq 0$**).**
For the continuous case with independence:

$$\boxed{p_{f/g}(z) = \int |v|\, p_f(z\cdot v)\, p_g(v)\, dv}$$

For the discrete case:

$$p_{f/g}(z) = \sum_{s\geq 1} s\cdot p_f(z\cdot s)\cdot p_g(s)$$

(The factor s is the Jacobian of the change of variable.)

Summary:

Operation	Formula (under independence)	Operation type	Complexity
$f+g$	$p_f * p_g$	Convolution	$O(L^2)$
$f-g$	$p_f * \tilde{p}_g$	Cross-correlation	$O(L^2)$
$f\cdot g$	$\sum_{rs=z} p_f(r)p_g(s)$	Divisor sum	$O(L^2 \log L)$
f/g	$\sum_s s\cdot p_f(zs)p_g(s)$	Scaling sum	$O(L^2)$

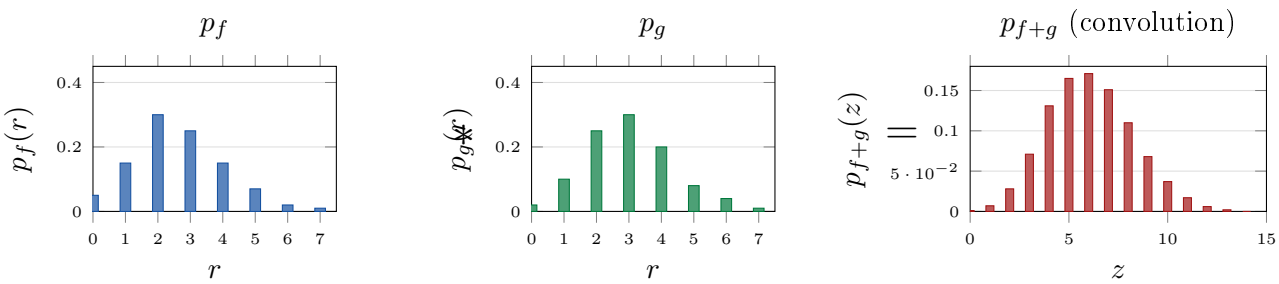


Figure 5: For independent images, the histogram of the sum $p_{f+g} = p_f * p_g$ (discrete convolution). The range doubles and the histogram broadens and smooths.

6 Problem 3.15 – Box-Filter Moving-Average Algorithm

Problem Statement

A lowpass filter can be implemented by setting all coefficients to 1, allowing use of a box-filter or moving-average algorithm.
(a) Formulate such an algorithm for an $n \times n$ filter, showing the nature of the computations and the scanning sequence.
(b) Obtain the computational advantage over a brute-force implementation and plot it as a function of n for $n > 1$. (Ignore the $1/n^2$ scaling factor.)

An $n \times n$ box (averaging) filter computes:

$$g(x, y) = \frac{1}{n^2} \sum_{s=0}^{n-1} \sum_{t=0}^{n-1} f(x + s,\, y + t)$$

Brute force at each (x, y): n^2 additions, n^2 fetches.
Moving-average insight: when the mask slides one step to the right (from column y to $y + 1$), only one column enters and one column leaves; the rest of the sum is unchanged. We only update the *part that changed*, avoiding recomputing the full sum from scratch.

Solution

(a) Algorithm.
Pre-computation – column sums (row integrals):
For every pixel column y and every starting row x, define:

$$C(x, y) = \sum_{i=0}^{n-1} f(x + i,\, y)$$

When the mask moves *down* by one row ($x \to x + 1$):

$$C(x + 1, y) = C(x, y) + f(x + n,\, y) - f(x,\, y) \quad \Rightarrow \quad \textbf{2 operations}$$

Main scan – sliding window in the horizontal direction:
For each row x, maintain the running horizontal sum:

$$S(x,\, 0) = \sum_{j=0}^{n-1} C(x,\, j) \quad \text{(initialise for first column)}$$

When the mask moves *right* ($y \to y + 1$):

$$S(x,\, y + 1) = S(x,\, y) + C(x,\, y + n) - C(x,\, y) \quad \Rightarrow \quad \textbf{2 operations}$$

Scanning sequence:

1. Scan row by row, left to right.

2. For the first position $(x_0, 0)$: compute $C(x_0, j)$ for $j = 0 \ldots n-1$ and sum them (n fetches and $n-1$ additions).

3. For subsequent positions: use the 2-operation updates above.

(b) Computational advantage.

- **Brute force (no $1/n^2$ factor):** n^2 additions per pixel.
- **Moving-average:** 2 additions per pixel for the horizontal sliding window + 2 additions for the column-sum update = 4 operations per pixel.

$$\boxed{\text{Computational Advantage} = \frac{n^2}{4}}$$

(For the simpler analysis of just one dimension of sliding – without pre-computing column sums – the update costs $2n$ ops per pixel, giving advantage $= n^2/(2n) = n/2$.) Either way, the advantage grows rapidly with n:

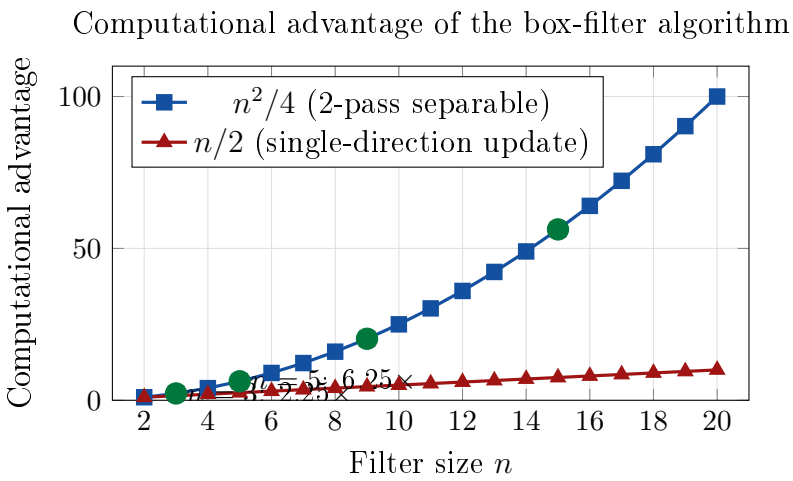


Figure 6: The computational advantage of the box-filter moving-average algorithm grows as $n^2/4$ (full 2-D separable update) or $n/2$ (single-direction update), making it highly efficient for large filters.

7 Problem 3.19 – Median Filter: Computation and Update

Problem Statement

(a) Develop a procedure for computing the median of an $n \times n$ neighbourhood.
(b) Propose a technique for updating the median as the centre of the neighbourhood is moved from pixel to pixel.

The **median filter** replaces each pixel with the *median* of the n^2 pixels in its $n \times n$ neighbourhood. Unlike the average filter, it is **non-linear** and is highly effective at removing *impulse (salt-and-pepper) noise* while preserving edges.
Why median? The median is the value that minimises the sum of absolute deviations; it is robust to outliers (a single noisy spike has no effect on the median of the neighbourhood if the neighbourhood is large enough).

Solution

(a) Computing the median of an $n \times n$ neighbourhood.

1. **Extract** all n^2 pixel values from the neighbourhood centred at (x, y) into a 1-D array A.
2. **Sort** A in non-decreasing order using any comparison sort (e.g. quicksort, heapsort).
   Time complexity: $O(n^2 \log n)$ per pixel.
3. **Select** the middle element:
   - If n^2 is *odd* (the usual case – n odd): median $= A[\lfloor n^2/2 \rfloor]$ (0-indexed).
   - If n^2 is *even*: median $= \frac{A[n^2/2-1]+A[n^2/2]}{2}$.

Example (3×3): $n^2 = 9$, median $= A[4]$ (5th element of the sorted 9-element array).

(b) Updating the median as the mask moves.
Motivation: Naïvely re-sorting after every step costs $O(n^2 \log n)$ per pixel. By tracking a **histogram** of the neighbourhood values, we can update in $O(n + \Delta)$ time where Δ is a small shift in the median.
Histogram-based update algorithm (Huang et al., 1979):
State maintained:

- $H[v]$: histogram of the n^2 values currently in the window.
- m: current median value.
- L_{cnt}: count of pixels with value $\leq m$ (the "left count").

When the mask moves right by one pixel ($y \to y + 1$, so column y is removed and column $y + n$ is added):

1. For each of the n removed pixels p_{out}:
   - Decrement $H[p_{\text{out}}]$.
   - If $p_{\text{out}} \leq m$: decrement L_{cnt}.
2. For each of the n added pixels p_{in}:
   - Increment $H[p_{\text{in}}]$.
   - If $p_{\text{in}} \leq m$: increment L_{cnt}.
3. *Adjust the median:*
   - While $L_{\text{cnt}} - H[m] > \lfloor n^2/2 \rfloor$: decrement m, update L_{cnt}.
   - While $L_{\text{cnt}} \leq \lfloor n^2/2 \rfloor$: increment m, update L_{cnt}.

Complexity: Step 1+2: $O(n)$. Step 3: $O(\Delta)$ where Δ is the change in the median (usually small).
Complexity comparison:

Method	Per-pixel cost	For $n = 7$ (49 pixels)
Brute-force sort	$O(n^2 \log n)$	~270 ops
Histogram update	$O(n)$	~14 ops

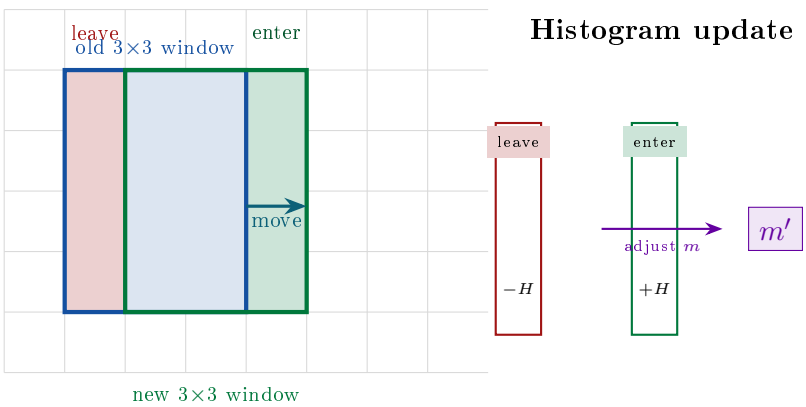


Figure 7: Moving the 3×3 median mask one step right: the leftmost column (red) is removed and the rightmost column (green) is added. Only $n = 3$ histogram entries need updating, and the median is adjusted by a small shift.

8 Problem 3.24 – Isotropy (Rotation Invariance) of the Laplacian

Problem Statement

Show that the Laplacian defined in Eq. (3.6-3) is isotropic (invariant to rotation). You will need the following equations relating coordinates for axis rotation by an angle θ:

$$x = x' \cos\theta - y' \sin\theta, \qquad y = x' \sin\theta + y' \cos\theta$$

where (x, y) are the unrotated and (x', y') are the rotated coordinates.

An image operator is **isotropic** if its response is *identical regardless of the direction in which a feature is oriented.* For a spatial filter, this means rotating the image and then applying the filter gives the same result as applying the filter and then rotating.
Eq. (3.6-3) – the Laplacian:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \tag{4}$$

The Laplacian is the **sum of the two pure second-order partial derivatives** – no mixed term $\partial^2/\partial x\,\partial y$, which is what enables the isotropic property.

Solution (Proof)

Goal: show that $\nabla^2_{x',y'} f = \nabla^2_{x,y} f$, i.e.

$$\frac{\partial^2 f}{\partial x'^2} + \frac{\partial^2 f}{\partial y'^2} = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Step 1 – First partial derivatives via chain rule.

$$\frac{\partial f}{\partial x'} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial x'} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial x'} = \cos\theta\,\frac{\partial f}{\partial x} + \sin\theta\,\frac{\partial f}{\partial y}$$

$$\frac{\partial f}{\partial y'} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial y'} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial y'} = -\sin\theta\,\frac{\partial f}{\partial x} + \cos\theta\,\frac{\partial f}{\partial y}$$

Step 2 – Second partial derivatives.
Applying the chain rule a second time:

$$\frac{\partial^2 f}{\partial x'^2} = \cos^2\theta\,\frac{\partial^2 f}{\partial x^2} + 2\cos\theta\sin\theta\,\frac{\partial^2 f}{\partial x \partial y} + \sin^2\theta\,\frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial y'^2} = \sin^2\theta\,\frac{\partial^2 f}{\partial x^2} - 2\cos\theta\sin\theta\,\frac{\partial^2 f}{\partial x \partial y} + \cos^2\theta\,\frac{\partial^2 f}{\partial y^2}$$

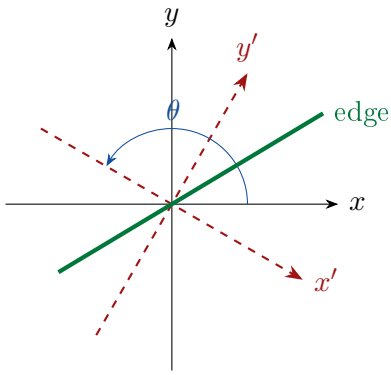
Step 3 – Add the two second derivatives.

$$\frac{\partial^2 f}{\partial x'^2} + \frac{\partial^2 f}{\partial y'^2} = (\cos^2\theta + \sin^2\theta)\frac{\partial^2 f}{\partial x^2} + \underbrace{(2\cos\theta\sin\theta - 2\cos\theta\sin\theta)}_{=0}\frac{\partial^2 f}{\partial x \partial y} + (\sin^2\theta + \cos^2\theta)\frac{\partial^2 f}{\partial y^2}$$

$$= 1 \cdot \frac{\partial^2 f}{\partial x^2} + 0 + 1 \cdot \frac{\partial^2 f}{\partial y^2} = \nabla^2 f \qquad \square$$

Conclusion. The Laplacian is invariant to rotation: rotating the coordinate system by any angle θ does not change the value of $\nabla^2 f$. Therefore it treats *all orientations equally* and is isotropic.

Key insight. The cross-derivative terms $\partial^2 f/\partial x \partial y$ cancel because they appear with equal but opposite signs in $\partial^2/\partial x'^2$ and $\partial^2/\partial y'^2$. This is a consequence of using the *sum* of the two pure second derivatives.



Rotating the coordinate axes by θ
does not change $\nabla^2 f$ at any point.

Figure 8: Under rotation of axes $(x, y) \to (x', y')$ by angle θ, the Laplacian is unchanged. This means a the Laplacian-based edge detector responds equally to edges of any orientation.

9 Problem 3.28 – Laplacian Subtraction is Proportional to Unsharp Masking

Problem Statement

Show that subtracting the Laplacian from an image is proportional to unsharp masking. Use the definition for the Laplacian given in Eq. (3.6-6).

A. Unsharp masking (traditional photography technique):

$$g_{\text{mask}}(x,y) = f(x,y) - \bar{f}(x,y) \quad \Rightarrow \quad g(x,y) = f(x,y) + k \cdot g_{\text{mask}}(x,y) \tag{5}$$

where $\bar{f}$ is a *blurred* version of f and $k > 0$ controls the degree of sharpening.

B. Laplacian sharpening (Eq. 3.6-6):
Eq. (3.6-6) is the **4-connected discrete Laplacian**:

$$\nabla^2 f(x,y) = f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1) - 4f(x,y) \tag{6}$$

The sharpened image:

$$g(x,y) = f(x,y) - c\,\nabla^2 f(x,y) \qquad c > 0 \tag{7}$$

(negative sign because the mask in Eq. (6) has a negative centre coefficient.)

Solution (Proof)

Step 1 – Choose the natural local average for Eq. (3.6-6).
The 4-connected Laplacian (Eq. (6)) involves the centre pixel and its 4 orthogonal neighbours. The *natural neighbourhood average* that exactly corresponds to this operator is the 5-element *plus-shaped* average:

$$\bar{f}(x,y) = \frac{1}{5}\Big[f(x,y) + f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1)\Big]$$

Step 2 – Compute the unsharp mask.

$$\begin{aligned}
g_{\text{mask}}(x,y) &= f(x,y) - \bar{f}(x,y) \\
&= f(x,y) - \frac{1}{5}\Big[f(x,y) + f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1)\Big] \\
&= \frac{4}{5}f(x,y) - \frac{1}{5}\Big[f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1)\Big] \\
&= -\frac{1}{5}\Big[\underbrace{f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1) - 4f(x,y)}_{\nabla^2 f(x,y) \text{ by Eq. (6)}}\Big]
\end{aligned}$$

Therefore:

$$\boxed{g_{\text{mask}}(x,y) = -\frac{1}{5}\,\nabla^2 f(x,y)} \tag{8}$$

Step 3 – Substitute into the unsharp masking formula.

$$
\begin{aligned}
g(x,y) &= f(x,y) + k \cdot g_{\text{mask}}(x,y) \\
&= f(x,y) + k \cdot \left(-\tfrac{1}{5}\nabla^2 f(x,y)\right) \\
&= f(x,y) - \underbrace{\frac{k}{5}}_{=c} \nabla^2 f(x,y)
\end{aligned}
$$

This is exactly the Laplacian-sharpening formula (Eq. (7)) with $c = k/5$.

Conclusion. Unsharp masking with the natural 5-element plus-shaped local average is *proportional to* (in fact, identical in form to) subtracting the Laplacian from the image. The proportionality constant is 1/5:

$$
g_{\text{mask}} = -\frac{1}{5}\nabla^2 f \quad \Longrightarrow \quad \text{unsharp masking} \propto \text{Laplacian sharpening} \qquad \square
$$

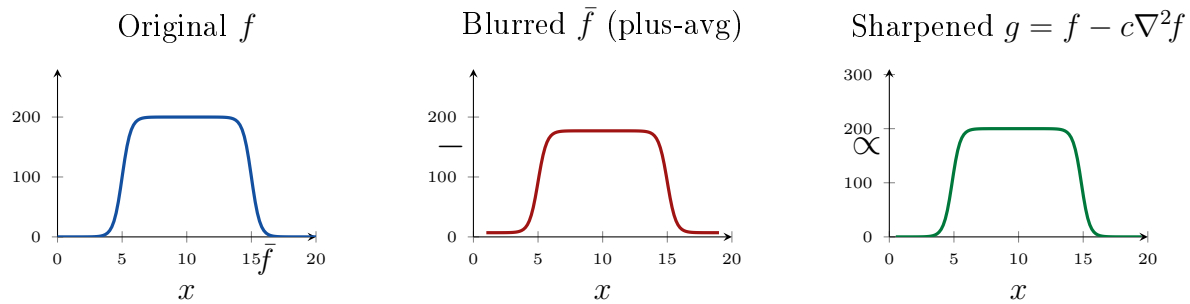


Figure 9: Laplacian sharpening enhances edges by amplifying the difference $f - \bar{f}$ which equals $-\frac{1}{5}\nabla^2 f$. The sharpened image (right) shows overshoot at transitions, characteristic of both unsharp masking and Laplacian sharpening.

Summary of All Solutions

Problem	Topic	Key Result
3.4	Bit-plane slicing	$T_k(r) = 255 \cdot (\lfloor r/2^k \rfloor \bmod 2)$; period $= 2^{k+1}$
3.5	Zeroing bit planes	Lower planes $\to$ histogram spikes at multiples of 2^p; higher planes $\to$ histogram folds into low range
3.6	HE non-flatness	Integer rounding merges adjacent levels (peaks) and leaves gaps (empty bins); exact uniformity is impossible
3.7	HE idempotency	Second CDF equals first: $T_2(s_m) = s_m$ (identity); HE applied twice = HE applied once
3.13	Combined histograms	Under pixel independence: $+ \to$ convolution, $- \to$ cross-corr., $\times/\div \to$ scaling sums
3.15	Box filter algorithm	Update: 2 ops/pixel; advantage $= n^2/4$ (2-pass) or $n/2$ (single-direction)
3.19	Median filter	Sort n^2 values: $O(n^2 \log n)$; histogram update when sliding: $O(n)$ per step
3.24	Laplacian isotropic	$\nabla^2_{x',y'} f = \nabla^2_{x,y} f$ because $\cos^2\theta + \sin^2\theta = 1$ cancels rotation
3.28	Laplacian $\propto$ unsharp	$g_{\text{mask}} = -\frac{1}{5}\nabla^2 f$; Laplacian sharpening $\equiv$ unsharp masking with $c = k/5$